AgriGraph AI

Technical Documentation

STGNN Pipeline, Backend API, and Frontend

Generated from repository source layout. Version 0.1.0.

Abstract

This document describes the AgriGraph AI prototype: the spatio-temporal graph neural network (STGNN) used for county-level sustainability scores, the Python/FastAPI backend (REST API, data flow, and training artifacts), and the React/Vite frontend (tabs, main user flows, and configuration). It is intended for developers and technical readers.

Contents

1 Scope and data context **3**

2 End-to-end pipeline overview **3**

2.1 Phase sequence . . . 3

2.2 Conceptual flow diagram . . . 4

3 Spatial graph (Phase A6) **4**

3.1 Construction . . . 4

3.2 Artifacts . . . 4

4 STGNN pipeline (Phases A7 and A7b) **4**

4.1 Libraries . . . 4

4.2 Node features and temporal tensor . . . 4

4.3 Targets and scaling . . . 5

4.4 Architecture . . . 5

4.5 Training logs . . . 5

4.6 Hyperparameters and optimisation . . . 5

4.7 Phase A7 — evaluation protocol . . . 6

4.8 Phase A7b — forecast weights . . . 6

4.9 Key artifact filenames . . . 7

5 Upstream modelling (CVAE and PCA pillars) 7
5.1 Conditional VAE augmentation (Phase A3) . . . 7
5.2 PCA pillars and scores (Phases A4–A5) . . . 7

6 Downstream of the STGNN: forecast, uncertainty, counterfactuals 8
6.1 Autoregressive forecast (Phase A8) . . . 8
6.2 Monte Carlo uncertainty (Phase A9) . . . 8
6.3 Counterfactual simulation (Phase A10) . . . 8

7 Backend: technologies and behaviour 8
7.1 Stack and dependencies . . . 8
7.2 Application entrypoint . . . 9
7.3 Loaded application state . . . 9
7.4 REST API summary . . . 9
7.5 Request bounds (illustrative) . . . 10
7.6 Forecast baselines (`/comparison`) . . . 10

8 Frontend: technologies and usage 10
8.1 Stack . . . 10
8.2 API configuration . . . 11
8.3 Main UI structure . . . 11
8.3.1 Atlas . . . 11
8.3.2 Benchmarks . . . 11
8.3.3 Methodology . . . 11
8.4 Typical local workflow . . . 11
8.5 Atlas tab (primary workflow) . . . 12
8.6 Benchmarks tab . . . 12
8.7 Methodology tab . . . 12
8.8 Frontend module map (concise) . . . 12

9 Operational notes 12
9.1 Pillar display scaling (API) . . . 13
9.2 CORS and deployment . . . 13
9.3 Illustrative JSON payloads . . . 13

A Tensor shapes and attention mask 13

B Repository locations (reference) 14

C ICBF feature columns used in PCA 14

1 Scope and data context

The application ingests county-year dairy sustainability indicators (ICBF-style features), builds augmented training views, derives four PCA-based *pillars* and an overall score, constructs a fixed spatial graph over the **26 Irish counties**, and trains an STGNN to forecast scores and support uncertainty and counterfactual tooling exposed via a REST API and consumed by a single-page web client.

Throughout this document, **Phase Ax** labels refer to the sequential pipeline modules under `backend/pipeline/` orchestrated by `run_pipeline()`.

Table 1: Pipeline modules invoked by `run_pipeline()` (import names).

Phase	Python module (under `backend/pipeline/`)
A1	`data_loader.py`
A2	`preprocessing.py`
A3	`vae.py`
A4–A5	`pca_pillars.py`
A6	`graph.py`
A7–A7b	`stgnn.py`
A8	`forecast.py`
A9	`uncertainty.py`
A10	`counterfactual.py`

2 End-to-end pipeline overview

2.1 Phase sequence

Table 2 lists the research phases in execution order. The STGNN (**A7** evaluation training + metrics, **A7b** full-data retrain for forecasting) sits after graph construction and before downstream forecast export.

Table 2: Pipeline phases (high level).

Phase	Role
A1	Load and validate raw panel data
A2	Preprocessing / cleaning to standardized tables
A3	Year-conditioned CVAE training and feature augmentation (variants)
A4/A5	PCA pillars and sustainability scoring
A6	Spatial k-NN graph over counties (edge index for PyTorch Geometric)
A7 / A7b	STGNN: honest metrics then forecast weights
A8	Forecast horizons and exports
A9	Uncertainty bands
A10	Counterfactual scenarios

2.2 Conceptual flow diagram

Figure 1 summarises how preprocessing, augmentation, pillars, and the spatial graph feed the STGNN and downstream analytics.

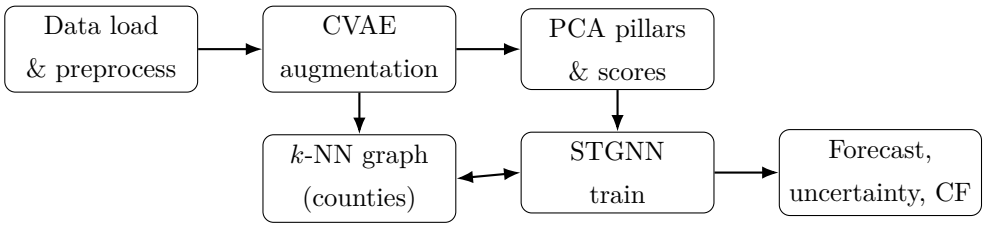


Figure 1: Abbreviated research pipeline: pillar scores and the county graph interact with STGNN training; subsequent phases consume the forecast-oriented model.

3 Spatial graph (Phase A6)

3.1 Construction

Counties are nodes ($N = 26$). Pairwise great-circle distances use the haversine formula on fixed centroid coordinates. A **k-nearest neighbour** adjacency is built per node (`K_NEIGHBORS` $= 3$), then **symmetrised**: an undirected edge exists if either endpoint lists the other among its k neighbours. Self-loops are removed.

3.2 Artifacts

Binary adjacency and a symmetric normalized Laplacian-style matrix are saved under `backend/artifacts/`; `edge_index.pt` stores directed edges for every undirected link (both directions) for PyTorch Geometric. County order is persisted in `graph_counties.pkl` and must stay aligned with STGNN node indexing.

4 STGNN pipeline (Phases A7 and A7b)

4.1 Libraries

Training and inference rely on:

- **PyTorch** (`torch`) — modules, Adam optimiser, automatic differentiation;
- **PyTorch Geometric** — `GCNConv` for graph convolution;
- **NumPy / pandas** — tabular IO and metric computation;
- **scikit-learn** — R^2, MAE, MSE on held-out predictions.

4.2 Node features and temporal tensor

For each variant ID and each input year in `INPUT_YEARS` $= [2021, 2022, 2023, 2024]$, the model builds a snapshot tensor of shape (N, F) with $F = 5$:

- Four pillar dimensions (from augmented pillar scores);
- One **lag-normalised** prior-year score channel (computed from sustainability scores with global lag statistics `y_bar`, `sigma_y`).

The full input to the network is therefore $X \in \mathbb{R}^{T \times N \times F}$ with $T = 4$, $N = 26$, $F = 5$.

4.3 Targets and scaling

The regression target is the county sustainability **score for year 2025** (per variant). Targets are **standardised** using the mean and standard deviation of scores computed from the training frames (saved as `stgnn_target_scaler.pkl`). Predictions are inverted to the original score scale for evaluation and reporting.

4.4 Architecture

The class `STGNN` (`backend/pipeline/stgnn.py`) implements:

1. **Spatial stack (per timestep)**: two `GCNConv` layers: F_{in}=5 $\rightarrow$ 64 $\rightarrow$ 64, each followed by ReLU and dropout (p=0.2).
2. **Temporal pooling**: for each timestep t, node embeddings $h_t \in \mathbb{R}^{N \times 64}$ are projected to queries, keys, and values in dimension d=32. Attention uses the **last** timestep's query against all timesteps' keys; a **causal** mask zeros out contributions from future timesteps so the final step cannot attend upward in time within the window.
3. **Head**: linear 32 $\rightarrow$ 32, ReLU, then 32 $\rightarrow$ 16 $\rightarrow$ 1 producing one scalar per county (shape $(N,)$).

4.5 Training logs

Intermediate diagnostics print at epochs `LOG_EPOCHS` = $(1, 100, 250, 500)$ and always at the final epoch (500). Logs report phase tags such as `A7-fold1`..`A7-fold4`, `A7-final`, and `A7b`.

4.6 Hyperparameters and optimisation

Table 3 lists the main constants in code.

Table 3: STGNN training hyperparameters (from `CONSTANTS` in `stgnn.py`).

Symbol / key	Value
`GCN_HIDDEN`	64
`ATTN_DIM`	32
`HEAD_HIDDEN`	16
`DROPOUT`	0.2
`EPOCHS`	500
`LEARNING_RATE`	10^{-3}
`WEIGHT_DECAY`	10^{-4}
`SEED`	42
`INPUT_YEARS`	2021–2024 (4 timesteps)
`TARGET_YEAR`	2025
`REAL_VARIANT_ID`	0 (non-augmented)
`SYNTHETIC_VARIANTS`	$\{1, 2, 3, 4\}$

The optimiser is **Adam** with parameters above. The training loss is **mean squared error** averaged over all training variants used in the step: for each variant v, the model predicts $\hat{y}_v \in \mathbb{R}^N$ and minimises $\|\hat{y}_v - y_v\|^2$ aggregated across variants and nodes (exact averaging matches the implementation in `_train_one_run`).

4.7 Phase A7 — evaluation protocol

- **k-fold on synthetic variants**: each of the four synthetic variant IDs is held out once as validation while the other three train (same epoch budget each fold). Training and validation R^2 are computed **on scaled targets**, pooled over nodes and variants in the split.
- **Final eval model**: train on *all* synthetic variants $\{1, 2, 3, 4\}$, then evaluate on the **real** variant (0) for year 2025 only — metrics reported as test R^2, MAE, MSE (and normalised MAE/MSE). This is the honest generalisation signal for the UI metrics endpoint.
- **Outputs:** `models/stgnn_eval_weights.pt`, `artifacts/stgnn_metrics.json`, `artifacts/stgnn_eval_predictions.csv`.

4.8 Phase A7b — forecast weights

A second model is trained on **all** variants $\{0, 1, 2, 3, 4\}$ with the same architecture and epochs. **No additional test metrics** are computed for this run; its weights drive operational forecasting and downstream phases. Output: `models/stgnn_forecast_weights.pt`.

Important: Do not report A7b training as "test performance" — there is no held-out year for that fit.

4.9 Key artifact filenames

File	Purpose
`edge_index.pt`	PyG edge list for GCN
`stgnn_forecast_weights.pt`	Deployed STGNN state dict + metadata
`stgnn_eval_weights.pt`	Eval-focused weights for metrics lineage
`stgnn_metrics.json`	A7 fold and test metrics
`stgnn_target_scaler.pkl`	Target mean/std for (de)normalisation
`lag_norm.pkl`	Lag-channel normalisation stats

5 Upstream modelling (CVAE and PCA pillars)

5.1 Conditional VAE augmentation (Phase A3)

The CVAE expands each real county-year row into additional synthetic *variant* rows (`variant_id` $\in \{1, 2, 3, 4\}$) while preserving year conditioning. Training uses PyTorch `nn` modules, Adam optimiser, batch size 32, β=1, latent dimension 80, hidden width 128, and 200 epochs with fixed seed 42. Synthetic diversity is checked against quality thresholds on mean absolute feature drift and correlation-matrix distance (`MEAN_ABS_DIFF_THRESHOLD` = 0.05, `CORRELATION_NORM_THRESHOLD` = 7.0). Primary artefacts: `backend/models/cvae_weights.pt`, `backend/data/x_augmented.csv`, and JSON logs under `backend/artifacts/`.

Table 4: Selected CVAE configuration keys (`vae.py`).

Key	Value
`INPUT_DIM`	17 herd features
`CONDITION_DIM`	5 (year one-hot + metadata)
`LATENT_DIM`	80
`HIDDEN_DIM`	128
`EPOCHS`	200
`LEARNING_RATE`	10^{-3}
`BATCH_SIZE`	32
`SYNTHETIC_PER_REAL`	4

5.2 PCA pillars and scores (Phases A4–A5)

Seventeen standardised features feed PCA; **four** components are retained as pillars (`N_COMPONENTS_RETAINED` equals four). Pillar interpretation uses loadings with a **75th-percentile** dominant-feature rule; some features are marked negative-polarity so that "higher is better" aligns across indicators. Scores are mapped to $[0, 100]$ for presentation. Frozen transforms (`standard_scaler.pkl`, `pca_loadings.npy`, `pillar_weights.pkl`, `score_params.pkl`) are consumed at API time for live pillar preview and counterfactual paths. The full feature set is listed in Appendix C.

6 Downstream of the STGNN: forecast, uncertainty, counterfactuals

6.1 Autoregressive forecast (Phase A8)

Phase A8 reloads the **forecast** STGNN checkpoint (`stgnn_forecast_weights.pt`) and rolls the model forward year by year from 2026 to 2036. It builds each year's input window from historical and extrapolated pillar trajectories, applying percentile-based guards on pillars, lags, and score dynamics, and optional out-of-distribution (OOD) blending so extreme extrapolations are damped. Rolling windows use `WINDOW_INPUT_YEARS` $= [2022, 2023, 2024, 2025]$ as the four-step input after the first forecast year. OOD detection combines normalised pillar, lag, and score signals with weights 0.45/0.25/0.30; blend factors `BLEND_WEIGHT_LOW/MEDIUM/HIGH` default to 0.85/0.55/0.25 for low/medium/high OOD scores. Outputs: `forecast_2026_2036.csv` and `forecast_pillars_2026_2036.csv`.

6.2 Monte Carlo uncertainty (Phase A9)

Uncertainty bands are generated by adding Gaussian noise in **score space** to the A8 forecast and taking empirical 5th/95th percentiles per county-year. Key parameters: `MONTE_CARLO_TRIALS` $=$ 100, `SIGMA_MC` $= 0.1$, random seed 42, with scores clipped to $[0, 100]$ after perturbation. Output: `uncertainty_bands.csv`, merged by the `/forecast` API.

6.3 Counterfactual simulation (Phase A10)

The counterfactual engine (`counterfactual.py`) composes user intervention vectors with the same frozen transforms and STGNN-based dynamics used in training, returning baseline and scenario score paths, per-pillar shifts, and short natural-language explanations (`backend/api/explanation.py`). Endpoints: `/counterfactual` and `/counterfactual/global`.

7 Backend: technologies and behaviour

7.1 Stack and dependencies

Table 5 lists pinned Python packages from `backend/requirements.txt`.

Table 5: Backend Python dependencies (versions as in repository).

Package	**Role**
FastAPI	HTTP API framework, routing, request validation
Uvicorn	ASGI server
pandas / NumPy / SciPy	Data frames, numerics
scikit-learn	PCA, scaling, metrics
PyTorch	Deep learning (CVAE, STGNN)
torch-geometric	Graph convolution layers
python-multipart	Form/file uploads if needed

7.2 Application entrypoint

`backend/main.py` constructs a FastAPI app with:

- **Lifespan startup**: if `stgnn_forecast_weights.pt` or `data/forecast_2026_2036.csv` is missing, it runs the **full offline pipeline** (`run_pipeline()`). Otherwise it skips training and only loads cached state.
- **CORS**: allows origin `http://localhost:5173` (Vite dev server).
- **Cache headers**: `Cache-Control: no-store` on `/api` responses to avoid stale JSON after regenerating artifacts.
- **Routes**: API router mounted at prefix `/api`; health check at `/health`.

7.3 Loaded application state

`load_state()` (`backend/api/state.py`) builds an `AppState` containing: frozen PCA/pillar transforms, county list, historical and forecast score tables, uncertainty bands, augmented and forecast pillar panels, normalised feature table for slider previews, STGNN metrics JSON, and human-readable pillar label keys for API responses.

7.4 REST API summary

Table 6 lists the main HTTP endpoints. All paths below are relative to the API base `/api`; the health route is an exception.

Table 6: REST API endpoints exposed by the FastAPI application.

Path	Method	Description
`/health`	GET	Liveness (`{"status":"ok"}`); mounted at app root, not under `/api`.
`/counties`	GET	Sorted list of valid county names.
`/historical`	GET	Historical county-year scores.
`/forecast`	GET	Forecast scores merged with 5th/95th uncertainty bands.
`/pillars/{county}`	GET	Pillar time series (historical + forecast) with normalised display columns.
`/pillars/preview`	POST	Instant pillar change from intervention deltas (frozen PCA transform).
`/counterfactual`	POST	County simulation: baseline vs scenario trajectories, pillar shifts, explanation text.
`/counterfactual/global`	POST	Multi-county scenarios for coordinated simulation.
`/comparison`	GET	STGNN vs linear-trend vs county-mean baselines.
`/metrics`	GET	Aggregated A7 test and fold metrics.
`/county/{county}/indicators`	GET	Score plus selected raw indicators (optional query `year`).
`/methodology/learning_curves`	GET	CVAE history JSON + STGNN fold R^2 pairs.
`/methodology/cvae_validation`	GET	Real vs synthetic summary stats for methodology UI.

7.5 Request bounds (illustrative)

Pillar preview applies bounded multiplicative deltas to normalised feature vectors (`DELTA_MIN` $= -1$, `DELTA_MAX` $= 1$), then clips to $[0, 1]$ before the frozen pillar transform. Counterfactual requests accept a `horizon` (default long horizon in single-county; global default often 11 years in line with UI).

7.6 Forecast baselines (`/comparison`)

The endpoint `/comparison` returns three trajectories per county-year for 2026–2036: **STGNN** forecasts from cached CSVs, a **linear trend** baseline (per-county linear regression over 2021–2025 extended forward), and a **county mean** baseline that repeats each county's 2025 score flat across the horizon. These baselines are for comparative analytics only and do not replace the operational STGNN forecast path.

8 Frontend: technologies and usage

8.1 Stack

- **React 18** with **Vite** as the dev server and bundler;

- **Tailwind CSS** for utility-first styling (including dark mode toggle on the root document);
- **Recharts** for time series, bands, and radar charts;
- **react-simple-maps** + **topojson-client** for the Ireland county map;
- **React Router** is a dependency (routing usage is minimal in the scaffold but available for expansion).

8.2 API configuration

The client expects the backend at `http://localhost:8000`. All JSON requests use the base URL defined in `frontend/src/config.js`:

```
export const API_BASE_URL = "http://localhost:8000/api";
```

Change this if you proxy through another host or deploy behind TLS.

8.3 Main UI structure

The shell in `App.jsx` defines three tabs: **Atlas**, **Benchmarks**, **Methodology**. Shared props include the county list, selected county/year, historical and forecast rows fetched at startup, and (for Atlas/Benchmarks) global counterfactual simulation state.

8.3.1 Atlas

Primary exploratory view: map-based county selection, sustainability trajectory with forecast band, pillar radar and per-pillar charts, simulation sidebar (scenarios, sliders, pillar preview), and optional counterfactual result layout after running the global simulation.

8.3.2 Benchmarks

County ranking and comparison views using merged timelines and KPI-style summaries; can link behaviour back to Atlas via props (e.g. confirming county focus).

8.3.3 Methodology

Static and data-driven explanation of the pipeline: learning curves and CVAE validation fetched from `/methodology/*` endpoints.

8.4 Typical local workflow

1. Install backend dependencies once: `pip install -r backend/requirements.txt` inside your Python environment.
2. Start API from repository root: `uvicorn backend.main:app`. Use `-reload` only when editing Python sources (it watches files and restarts the server).
3. Install frontend packages once: `cd frontend && npm install`.
4. Start Vite: `npm run dev`; open `http://localhost:5173`.

5. Verify `GET http://localhost:8000/health`. If `stgnn_forecast_weights.pt` or `forecast_2026_2036.csv` is missing, startup triggers `run_pipeline()` and may run for a long time on first boot.

8.5 Atlas tab (primary workflow)

1. Wait for counties and base timelines to load (spinner clears).
2. Select a county on the map or list; optionally adjust the analysis year on any chart control that exposes it.
3. Inspect sustainability trajectory (historical vs forecast with uncertainty ribbon where applicable) and pillar radar / per-pillar trends.
4. Open the simulation sidebar: pick scenario chips or edit sliders representing herd interventions.
5. Use pillar preview (POST `/pillars/preview`) for immediate feedback; run the global counterfactual when ready — the app batches scenarios and POSTs to `/counterfactual/global`.
6. Switch simulation result toggles to compare baseline vs intervention layouts when results return.

8.6 Benchmarks tab

Use this tab for cross-county comparison views and KPI rankings fed by the same cached forecast/historical payloads; simulation hooks may delegate focus back to Atlas depending on UI state.

8.7 Methodology tab

Static narrative plus charts sourced from `/methodology/learning_curves` (CVAE loss history and STGNN fold R^2 bars) and `/methodology/cvae_validation` (real vs synthetic moments).

8.8 Frontend module map (concise)

Area	Role
`src/api.js`	Fetch wrappers, timeline helpers, score tiers
`src/views/AtlasView.jsx`	Map + dashboard + simulation orchestration
`src/views/BenchmarksView.jsx`	Rankings and comparative charts
`src/views/MethodologyView.jsx`	Pipeline narrative and methodology charts
`src/components/PillarRadar.jsx`	Four-pillar radar chart
`src/components/atlas/*`	Map, sidebars, charts, simulation toggles

9 Operational notes

- **Single source of truth for metrics**: use `/api/metrics` and `artifacts/stgnn_metrics.json` for A7 test numbers only.
- **Forecast weights**: production forecasting and uncertainty pipelines must consume `stgnn_forecast_weights.pt`, not the eval checkpoint.
- **Reproducibility**: STGNN training fixes NumPy and PyTorch RNG seeds to 42 (CPU training in code paths inspected).

9.1 Pillar display scaling (API)

Endpoints returning pillar rows attach normalised display columns by clipping each pillar z-score to $[-3, 3]$ then linearly mapping to $[0, 100]$ with unit spacing proportional to a six-point span (`routes.py`). These columns power radar charts; underlying PCA values remain in statistical units.

9.2 CORS and deployment

The API permits browser requests only from `http://localhost:5173`. Deploying the frontend elsewhere requires updating `FRONTEND_ORIGIN` in `backend/main.py` and `API_BASE_URL` in `frontend/src/config.js` consistently. Reverse proxies should forward `/api` and disable caching for JSON (the middleware already sets no-store for `/api`).

9.3 Illustrative JSON payloads

Pillar preview (`POST /api/pillars/preview`): intervention keys must match engineered feature names (see Appendix C). Example body:

```
{
  "county": "Tipperary",
  "interventions": { "calving_interval": -0.02 }
}
```

Single-county counterfactual (`POST /api/counterfactual`): includes optional long-range `horizon` (years simulated beyond baseline mechanics).
Global counterfactual (`POST /api/counterfactual/global`): expects `scenarios` as a map `{ "<County>": { "<feature>": <delta>, ... }, ... }` plus `horizon` for simultaneous multi-county runs.

A Tensor shapes and attention mask

Let T=4, N=26, F=5. The module receives $X \in \mathbb{R}^{T \times N \times F}$ and `edge_index` with shape $(2, E)$. After spatial GCN blocks, tensors $Q, K, V \in \mathbb{R}^{T \times N \times d}$ are formed with d=32. Attention logits for the final timestep use row $T-1$ of Q against all rows of K; entries corresponding to future timesteps are masked to $-\infty$ before softmax, enforcing causality within the window.

B Repository locations (reference)

Path	Contents
backend/main.py	FastAPI app, CORS, lifespan, router mount
backend/api/routes.py	REST endpoints and request models
backend/api/baselines.py	Linear trend and flat county baselines
backend/pipeline/	Phases A1–A10 modules
backend/data/	CSV inputs and generated panels
backend/models/	CVAE and STGNN weight files
backend/artifacts/	Metrics JSON, CVAE logs, graph matrices
frontend/src/	React application source

C ICBF feature columns used in PCA

The following 17 columns (see `pca_pillars.py` `FEATURE_COLS`) are standardised before PCA. Order is fixed in the pipeline.

- num_herds
- calving_interval
- six_wk_calving_rate
- calves_per_cow_per_year
- females_not_calved_pct
- avg_lactations
- current_replacement_rate
- potential_replacement_rate
- cows_culled_pct
- heifers_calved_22_26mo
- mortality_dead_at_birth
- mortality_dead_28days
- replacements_dairy_ai_pct
- births_known_sire_pct
- births_calving_survey_pct
- recycled_cows_pct
- autumn_6wk_calving_rate

Glossary (selected)

Variant ID Index 0 denotes observed (real) rows; 1–4 denote CVAE-derived synthetic replicates for the same county-year stratum.

Pillar One of four PCA-derived orthogonal summaries of herd indicators before interpretation labelling.

Lag channel Fifth STGNN input feature carrying normalised prior-year score inertia per county.

Eval weights STGNN checkpoint trained for held-out metrics on real 2025 scores (Phase A7 lineage).

Forecast weights STGNN checkpoint trained on all variants for operational rollout (Phase A7b).

OOD blend Mechanism in Phase A8 that reduces reliance on wild extrapolations when pillar/lag/score trajectories look inconsistent with historical envelopes.

Disclaimer

This documentation mirrors the current codebase structure. Numerical results depend on input CSVs and trained weights under `backend/data/` and `backend/models/`. Regenerating artefacts changes

forecasts and metrics.